



Comprehensive assessment of triclustering algorithms for three-way temporal data analysis

Diogo F. Soares^{a,*}, Rui Henriques^b, Sara C. Madeira^a

^a LASIGE, Faculdade de Ciências, Universidade de Lisboa, Portugal

^b INESC-ID and Instituto Superior Técnico, Universidade de Lisboa, Portugal

ARTICLE INFO

Dataset link: <https://github.com/dfmsoares/triclustering-algorithms-assessment>

Keywords:

Triclustering
Temporal data
Three-way data

ABSTRACT

The analysis of temporal data has gained increasing attention in recent years, aiming to identify patterns and trends that change over time. Temporal triclustering is a promising approach for this purpose, as it allows for the simultaneous clustering of three dimensions of data: objects, attributes, and time. In this work, we present a comparative study and experimental evaluation of state-of-the-art temporal triclustering algorithms. Our study provides a comprehensive quantitative assessment of several triclustering algorithms to unravel their strengths and limitations. To this end, we consider synthetic data with varying sizes and regularities, where true solutions are planted with different coherence and quality criteria, in order to assess the algorithms' performance in datasets with diverse characteristics and assess their capacity to retrieve specific types of hidden patterns. This provides a more comprehensive evaluation of the algorithms and allows for a better understanding of their capabilities and limitations. This study is the first to compare state-of-the-art triclustering algorithms inherently prepared to deal with temporal data and provides new benchmark results for the Temporal Triclustering task. Our results on the algorithms' performance can guide practitioners in selecting the most appropriate algorithm for their specific application.

1. Introduction

Three-way data (also referred to as three-dimensional, triadic, tensor, or cubic data), more concretely temporal three-way data, have an increasing prevalence in different fields of study as they offer a means to capture the behavior of systems over time [1]. In this context, in recent years, there has been a growing interest in using machine learning to analyze temporal data. Temporal data refers to data collected over time and can be found in various applications, such as finance, healthcare, and social media [1,2]. A critical task in analyzing temporal data is to identify patterns and trends that change over time. One way to do this is through the use of triclustering algorithms, which allow for the simultaneous clustering of three dimensions of data: objects, attributes, and time points.

In this work, we present a comparative study and experimental evaluation of triclustering algorithms inherently prepared to process temporal data, here referred as temporal triclustering algorithms. We first provide an overview of the existing literature on triclustering and the challenges that arise when applying it to temporal data. We then thoroughly compare several state-of-the-art temporal triclustering algorithms, pinpointing their strengths and limitations. Our study aims to provide insights into the performance of these algorithms and to

guide practitioners in the selection of the most appropriate algorithm for their specific application.

Additionally, since this study is the first, to our knowledge, to experimentally compare state-of-the-art algorithms in the field, our work will provide benchmark results for these methodologies that other researchers can use as a reference for future studies.

The contributions of this work are the following:

- A comprehensive comparison of state-of-the-art temporal triclustering algorithms, including their strengths and limitations.
- An experimental evaluation of the algorithms using synthetic datasets with true hidden solutions. This allows for a more comprehensive evaluation of the algorithms and provides a better understanding of their capabilities and limitations.
- First benchmark results for the Temporal Triclustering task, providing insights into the performance of the algorithms and guiding practitioners in selecting the most appropriate algorithm for their specific application.
- A deeper understanding of the performance of these triclustering algorithms, obtained by using ground truth data and performance

* Corresponding author.

E-mail address: dfsoares@ciencias.ulisboa.pt (D.F. Soares).

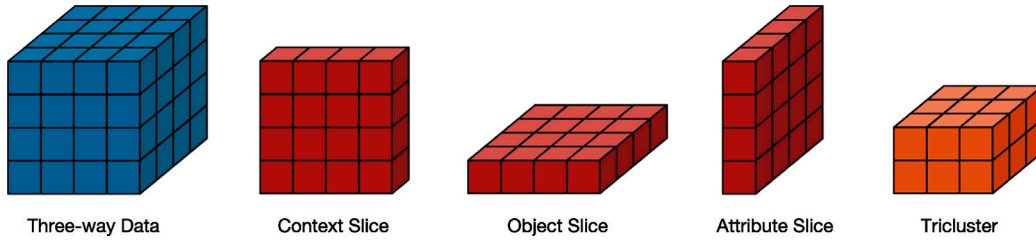


Fig. 1. Concepts used in three-way data analysis.
Source: Adapted from [1].

evaluation of extrinsic metrics, quantifying the impact of different pattern characteristics on the performance of the algorithms.

The paper is organized as follows: Section 2 introduces the problem formulation and key definitions for this study; Section 3 presents a review and characterization of the temporal triclustering algorithms presented in the literature; Section 4 describes the experimental methodology followed in this comparative study; Section 5 presents and discusses the results; and Section 6 concludes the paper and proposes future research directions.

2. Problem formulation

Three-way data. Three-way data (tridiac, three-dimensional, tensor, or cubic data) is defined as a dataset with n objects $X = \{x_1, \dots, x_n\}$, m attributes $Y = \{y_1, \dots, y_m\}$ and p contexts $Z = \{z_1, \dots, z_p\}$. Elements a_{ijk} relate object x_i , attribute y_j , and context z_k . Three-way data can be real-valued ($a_{ijk} \in \mathbb{R}$), symbolic ($a_{ijk} \in \Sigma$, where Σ corresponds to a set of ordinal or nominal symbols), or integer ($a_{ijk} \in \mathbb{N}$) [1].

When the contexts correspond to time points, we encounter a temporal three-way dataset, also known as a three-way time series. In this type of dataset, each object represents a multivariate time series with an order of m .

Tricuster. Given a 3W dataset H with n objects X , m attributes Y , and p contexts Z , a **tricuster** $t = (I, J, K)$ is a subspace of the original space, where $I \subseteq X$, $J \subseteq Y$, and $K \subseteq Z$ are subsets of objects, features, and contexts, respectively [1].

Triclustering solution. Given H , the triclustering task aims to find a set of triclusters $T = \{t_1, \dots, t_i\}$ such that each tricuster t_i satisfies specific criteria of homogeneity and statistical significance [1].

Fig. 1 provides a visual representation of the concepts used in 3W data analysis. In this figure, a tricuster is defined as subsets of objects that exist simultaneously within each of the three dimensions.

3. Algorithms

During the last decade, many algorithms have been proposed to address the challenges of finding subgroups concerning the three dimensions inherent in three-way data (objects, attributes, time). By analyzing the literature, we can identify different categories of algorithmic approaches. A survey on triclustering [1] categorizes the algorithms into six groups (greedy, stochastic, exhaustive, biclustering-based, pattern-based, multi-objective) that we refined here. Fig. 2 summarizes the five categories of algorithms we found in the literature for addressing triclustering concerning temporal data. We refined three of them into small subcategories, including algorithms with specific similar behavior. The criteria for the categories and corresponding subcategories' definitions were the following:

1. **Biclustering-based.** Algorithms that perform biclustering (similar to triclustering but in two-way data) in a first phase, leading to the discovery of triclusters using the biclusters identified in slices.

- (a) **Biclusters Enumeration.** Algorithms that enumerate all the solutions (either biclusters or triclusters directly) for the original problem. Typically these algorithms use a graph search enumeration process.
- (b) **Similarity-based.** These algorithms first search for biclusters in slices and then merge these biclusters searching for interplane coherence (similarities between planes) to obtain triclusters.

2. **Heuristic-search based.** Algorithms that follow iterative searches to find solutions.

- (a) **Divide-and-conquer.** Algorithms that split the problem into smaller problems and solve them individually, combining the solutions to achieve the final solution to the original problem.
- (b) **Greedy with seed growth.** Algorithms that randomly define a couple of seeds (small triclusters) and then construct the solution iteratively regardless of the impact of the decisions made at each iteration.

3. **Pattern-based.** Algorithms that perform pattern-based approaches to find the triclustering solution.

- (a) **Association Rule Mining (ARM) based.** These approaches use the concepts of frequent patterns to discover coherent patterns in three-way data, leading to the triclusters composition.
- (b) **Formal Concept Analysis (FCA) based.** FCA can be used to discover triclusters in binary three-way data via pattern extraction from a ternary relation between concepts. Although none of the algorithms in literature had been applied in temporal data, their potentialities can be extended to deal with this type of data.

4. **Stochastic.** Algorithms that make assumptions about the statistical distribution underlying the data trying to find triclusters according to these statistical models.

5. **Multi-objective Optimization.** Algorithms that use merit functions to optimize predefined objectives to be observed in triclustering solution, usually, concerning volume and homogeneity. Most of the algorithms in this category use evolutionary approaches, such as mutation, recombination and selection operations, to form triclusters.

In the following sections we briefly explain the ideas behind each identified algorithm for each category.

3.1. Biclustering-based

Temporal three-way data space can be seen as a set of two-way datasets (or simply matrices) corresponding to time points. Thus, biclustering can be applied to discover sets of biclusters for each slice of 3W data (temporal slice) from which a set of triclusters can be deduced (according to specific interplane coherence). The initial strategies considered here can be based on biclusters/triclusters enumeration.

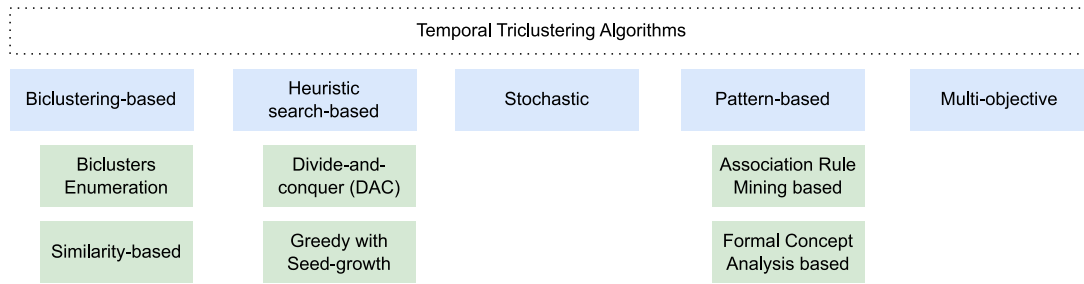


Fig. 2. Approaches to temporal triclustering.

These strategies entail exhaustively enumerating all potential solutions for a given problem, either by initially focusing on biclusters and subsequently inferring triclusters or by directly enumerating triclusters. On the other hand, the subsequent approaches considered are based on discovering biclusters for the slices of a given dimension and then merging the biclusters using a similarity-based metric to obtain the triclusters.

We start by identifying four algorithms that perform biclustering enumeration. **triCluster** [3] was the pioneer triclustering algorithm and was designed to deal with gene expression data. It first constructs a multigraph with similar ranges between all pairs of objects (genes) for each temporal slice in 3W data, following it searches for maximal cliques in the constructed multigraphs to find the sets of biclusters per time point; finally, it merges the similar biclusters from the search in a new multigraph where the nodes are now biclusters. **gTriCluster** [4] includes intertemporal coherence, which was a gap in the first considered algorithm, using Spearman correlation coefficient between time points. In terms of behavior, this algorithm is similar to the previous one, improving by using dedicated data structures (set enumeration trees) to mine the biclusters. **TD-Clustering** [5] aims to find time-delayed scaling, shifting or inverting triclusters in 3W temporal data. This algorithm performs a depth-first search through a tree structure to mine all maximal biclusters satisfying a minimum number of objects and time points according to a predefined value of coherence strength. **TCtriCluster** [2,6] is an extension of the original triCluster algorithm able to find time contiguous triclusters by imposing a coherence constraint of contiguity when merging biclusters from the multigraph, thus searching only for biclusters contiguous in time to be merged in the resulted triclusters.

The following four algorithms use a similarity-based strategy to merge biclusters into triclusters. **ICSM** [7] performs order-preserving biclustering in each context slice and merges the biclusters according to time dimension using a Planar Mean Residue Similarity (PMRS). **OPTriCluster** [8] finds triclusters with order-preserving coherence in the attribute slices, while removing contiguity in the time dimension the placed coherence ensures less sensitivity to variations. **THD-TriCluster** [9] identifies triclusters over gene-sample-time data with any absolute, shifting, scaling and shifting-and-scaling patterns, and high biological significance. It introduces a coherence measure (using Shifting-and-Scaling Similarity - SSSim) referred to as Inter-temporal to support effective clustering for the temporal domain. **POPTric** [10] uses a pathway-based approach to identify patterns in gene expression data while also preserving the order of genes, samples, and time points to identify subsets of genes that are co-regulated over time in specific biological pathways.

3.2. Heuristic-search based

Divide-and-conquer algorithms split the problem into smaller problems and solve each one individually to find the solution for the initial large problem. We considered two algorithms following this approach. **TRI-Clustering** [11] uses an Automatic Boundary Search (ABS) to unravel coherent triclusters with differential coherence in object slices.

It starts by generating random seeds and trims triclusters until the convergence (maximal iterations reached). **δ -TRIMAX** [12–14] finds triclusters with low mean square residue (MSR) in temporal 3W data. Initially, **δ -TRIMAX** considers the whole 3W dataset (objects, attributes, and contexts) as a candidate tricluster. At each iteration, some objects are deleted until the solution's MSR becomes lower than a predefined threshold ($MSR < \delta$). Then, new objects are added to the solution if they do not cause the MSR to exceed the δ threshold.

On the other hand, greedy approaches plant seeds (building blocks), instead of considering the whole 3W dataset at as a starting point. They start with small solutions that are gradually extended to form candidate triclusters. **MIC** [15] and **LagMiner** [16] are two examples of this type of approach. **MIC** aims to find statistically significant triclusters in temporal 3W data with an approximate constant coherence. To this end, **MIC** first generates seeds (pairs of correlated objects) with residual p-values. Then it grows correlated 3D subspace clusters (CSCs) from these seeds by greedily extending and merging seeds that maximize the proposed correlation scores. On its turn, with the same basis, **LagMiner** aims to find shift-and-scale correlated triclusters (S^2D^3 -Clusters) in gene-sample-time expression data. The found triclusters have to respect two predefined criteria: (1) shifting-and-scaling intraplane coherence for each time point (gene-sample slice) and object (time-gene slice), and (2) an order-preserving intraplane coherence within each sample (gene-time slice).

3.3. Pattern-based approaches

Specific, well-defined patterns in 3W subspaces can be mined using pattern-based triclustering approaches based on traditional association rule mining (ARM). We found five algorithms following this strategy. Ji et al. [17] propose a naive approach, referred to as **RSM**, that explores the frequent close patterns in data slices, merging them into frequent close cubes (FCC), and pruning the ones that are not closed. In the same work [17], **CubeMiner** directly operates on 3W data to discover FCCs splitting the data according to well-defined constraints. **3D-TDAR-Mine** [18] identifies triclusters given by Temporal Dependency Association Rules (TDAR) in temporal data. It does not require discretization, and the placed coherence is able to accommodate shifts, scales, and trends. It starts by computing patterns per object, where coherent time segments represent patterns for at least two attributes. Frequent pattern mining is applied to find the common patterns in multiple objects, and then using association rule mining is used to discover dependencies between patterns to compose triclusters. **Versatile Temporal Subspace discovery** [19] describes a tricluster by a clear temporal pattern given by the longest possible smooth sections of non-overlapping waveforms using the LOESS residue (computed from the lowest-order polynomial fitting the observed values). An exhaustive prefix-based search algorithm is then proposed with Pearson correlation (parameterized with the LOESS residue) to discover all maximal triclusters from closed temporal patterns. Due to the potential high number of triclusters, the top-k triclusters can be alternatively outputted. Mondal et al. [20] introduced **suffix trees to search for triclusters** [20]. The algorithm uses a suffix tree data structure to efficiently search for

triclusters, while also incorporating a heuristic approach to reduce the search space and avoid redundancies.

Another strategy included in this category is regarding **Formal Concept Analysis**. All the proposed algorithms based on Formal Concept Analysis deal with binary data, where each entry represents a relation between three concepts, termed triadic formal concepts. The main principle of this type of algorithm is to search all the possible pairs of objects and attributes exhaustively and then assemble them in triclusters. They use quality scores based on density, cardinality, coverage and diversity of subspaces to guide the search for coherent triclusters. Many algorithms (exploring several approaches) were designed in the last years [21–23], after the concept formalization made by Ignatov et al. [24]. Although none of the algorithms is able to handle temporal or real-valued data inherently, based on the work in [25], the underlying principles show sufficient evidence to be extended for triclustering temporal 3W data.

3.4. Stochastic

Stochastic algorithms assume that data follows a known statistical distribution and try to find the model's parameters that allow generating of the data under analysis. We identified four algorithms in this category. **Gene Program** [26] finds triclusters using Markov Chain Monte Carlo sampling to estimate a model's posterior probability distribution of subspaces after applying a discretization method to the 3D real-valued dataset. **MultiFacTV** [27] uses a tensor factorization objective function (holding an approximate constant coherence assumption with plaid effects), together with a time-axis regularization (enforcing subspaces to have objects as contiguous as possible in the time dimension). Triclusters are then derived from the factorized tensors. This algorithm requires the specification of the target number of triclusters and thresholding parameters per dimension. A triclustering plaid model, referred here as **3D-Plaid**, was introduced in [28], where the whole 3W data structure is seen as a set of overlapping triclusters from where the algorithm tries to iteratively find subspaces expressing high deviations verifying their dependence through minimizing the sum of squares of the 3W data. **TWIGS** [29] tries to find triclusters in 3D temporal data, where objects may be temporally misaligned. A hierarchical Bayesian model is used to find biclusters across object slices to then produce candidate triclusters. Gibbs sampling is applied to optimize the model's parameters.

3.5. Multiobjective

The triclustering task can be seen as a multi-objective optimization problem since we typically have different objectives clashing with each other, such as volume and homogeneity. It is a trade-off because the increased volume will lead to reduced homogeneity. All the algorithms included here have evolutionary approaches, having in their basis operations such as exchanges, mutations, and selections of objects. **MOGA3C** [30], **TriGen** [31] and **EMOA- δ -TRIMAX** [32] fall in this category. The pioneer algorithm following this approach was **MOGA3C** [30]. This algorithm aims to satisfy three simultaneous objectives: volume, homogeneity and object(gene)-dimension variance. **TriGen** [31], in its first version, is guided by two objectives: maximize triclusters' volume and minimize 3D mean square residue. TriGen was recently improved to yield a flexible form of coherence with two new merit functions: MSL [33], and LSL [34]. These new measures enable the possibility to accommodate time lags on object dimensions. **EMOA- δ -TRIMAX** [32] introduced a multiobjective search improving δ -TRIMAX [12] preventing overlapping while searching to maximize volume and minimize MSR 3D.

4. Triclustering evaluation

In this study, we assess state-of-the-art triclustering algorithms from the aforementioned categories: Biclustering-based, Heuristic-search based, and Multi-objective optimization. Current pattern-based approaches are not designed to address real temporal data properly, and the stochastic approaches force the definition of a strict number of patterns which can bias the evaluation. To this end, we evaluate each algorithm's solution against true (hidden) solutions using synthetic datasets with planted triclusters by G-Tric [35], a software tool to generate three-way datasets with hidden true triclustering solutions. An objective evaluation in real datasets would not be possible since no true solutions are available.

Horta and Campello introduced a set of desirable properties for the metric used to evaluate biclustering solutions [36]. Building upon their work, we now propose an extension of these criteria that evaluation metrics should satisfy in order to provide a comprehensive assessment of triclustering solutions:

- **Maximum**: an equivalent solution should have a maximum score;
- **Symmetry**: the order in which the subspace clusters are represented should not influence the outcome;
- **Dissimilarity**: penalizes solutions with redundant triclusters;
- **Precision**: penalizes solutions that use more than one subspace to cover a single tricluster;
- **Non-intersecting Area**: penalizes quality decays when a tricluster includes other tricluster's elements;
- **Coverage**: a solution is supposed to cover all planted subspaces; and
- **Statistical Significance**: ensures maximal solutions without spurious subspaces.

To this, we selected and extended three metrics from the literature for biclustering evaluation to assess triclustering solutions. Considering T , a triclustering a solution and T' a hidden true solution (ground truth):

1. The Clustering Error (CE), proposed in [37] measures the relationship between the number of non-intersecting elements from the triclustering solution and the ground truth solution considering the global space covered by them. Despite its simplicity, this metric met all the criteria defined before when assuming the planted triclusters are statistically significant. In this work, we adapted CE to three-way data, according to the previous definition of tricluster, thus being able to analyze the triclustering solutions discovered in the context of this work:

$$CE(T, T') = \frac{|U| - \sum_{t \in T} \max_{t' \in T'} r(t, t')}{|U|}, \quad (1)$$

$$|U| = |U_T \cup U_{T'}|, \quad (2)$$

$$r(t, t') = \frac{|t \cap t'|}{|t'|}, \quad (3)$$

$$U_T = \bigcup_{i \in T} I_i \times J_i \times K_i, \quad (4)$$

where $|U|$ is the number of elements covered by both solutions and U_S is the union of rows I , columns J , and contexts K of all the triclusters in the solution T .

2. Recoverability (R), measures the proportion of elements retrieved for each of the planted triclusters and it is defined as follows:

$$R(T, T') = \frac{\sum_{t \in T} \max_{t' \in T'} r(t, t')}{|T|}. \quad (5)$$

The goal is to maximize this metric, and the mean value for the recoverability in each tricluster can be taken as the solution's score, as defined in (5). Although unable to meet some of the

Table 1
Temporal triclustering algorithms used in the evaluation.

Algorithm	Link to source code
triCluster	https://github.com/zakimjz/TriCluster/tree/master/TriCluster
δ -Trimax	https://github.com/novalsaputra/Delta-Trimax
TriGen	https://github.com/davgutavi/trlab-application/releases/tag/v3.5
TCtriCluster	https://github.com/dfmsoares/triclustering-based-classifier/blob/main/src/TCtriCluster.py

previously defined criteria, it provides valuable feedback on the triclusters that compose the final solution, as it allows us to check what is being recovered and on which triclusters the algorithm is failing.

- 3D revised match score (RMS3) [1] is a Jaccard-based score extended to assess all the dimensions in triclustering, with the detail that it introduces a penalization for the non-matched elements between the triclustering solution and the ground truth, and is defined as follows:

$$\text{RMS3}(T, T') = \frac{1}{|T|} \sum_{t \in T, \max \arg\{\text{Jac}(t, t') | t' \in T'\}} \sqrt[3]{\frac{|t^I \cap t'^I|}{|t^I \cup t'^I|} \frac{|t^J \cap t'^J|}{|t^J \cup t'^J|} \frac{|t^K \cap t'^K|}{|t^K \cup t'^K|}}. \quad (6)$$

5. Methods

This section introduces the methodology followed to perform the comprehensive assessment and comparison of the state-of-the-art triclustering algorithms inherently prepared to deal with temporal data.

5.1. Algorithms

For this experimental evaluation, we considered triclustering algorithms from three of the categories defined before: Biclustering-based, Heuristic-search based, and Multi-objective optimization. We selected four algorithms for this study: triCluster, TCtriCluster, δ -trimax, and TriGen. These are those in the literature explicitly designed to tackle temporal 3W data analysis, whose implementations are available. [Table 1](#) lists the selected algorithms and the links to the public source code. The majority of the triclustering algorithms referred to in the literature have no implementations available. We attempted to reach out to the authors to obtain source codes from other algorithms but with limited success.

Table 2
Properties of the synthetic datasets used in the 5 evaluation settings.

	Properties	1st/2nd collection	3rd collection	4th collection	5th collection
Dataset	Data type	Real valued	Real valued	Real valued	Real valued
	Dimensions	$500 \times 10 \times 5$	$501 \times 10 \times 5$	$502 \times 10 \times 5$	$500 \times 10 \times 5$
	Alphabet	[0, 50 000]	[0, 50 000]	[0, 50 000]	[0, 50 000]
	Background	Uniform	Uniform	Uniform	Uniform
	Missings	0%	0%	0%	0%
	Noise	0%	0%	10%	0%
	Errors	0%	0%	5%	0%
Triclusters	Dimensions	$U(8,12) \times U(4,6) \times U(2,4)$	$U(8,12) \times U(4,6) \times U(2,4)$	$U(8,12) \times U(4,6) \times U(2,4)$	$U(8,12) \times U(4,6) \times U(2,4)$
	Contiguity	No	No	No	Yes (Contexts)
	Missings	0%	0%	0%	0%
	Noise	0%	0%	10%	0%
	Errors	0%	0%	5%	0%
	Noise deviation	0	0	2500	0
Overlapping	Plaid coherence	No	Additive/Multiplicative (M)	No	No
	% Overlapping trics	0%	40%	0%	0%
	Max. Interactions	0	2	0	0
	% Overlapping Elems	0%	50%	0%	0%
	Restrictions on	0%/0%/0%	100%/100%/100%	0%/0%/0%	0%/0%/0%
	rows/columns/contexts				

5.2. Data collections

Evaluating triclustering algorithms is challenging due to the absence of public datasets with known triclustering solutions, hindering the comparison of different algorithms and the assessment of each approach's strengths and weaknesses. Lobo et al. proposed G-Tric [35] to address this problem by presenting a tool for generating synthetic three-way datasets with triclustering solutions. This was the first proposal to tackle a key lack for triclustering evaluation and remains the state-of-the-art.

This three-way data generator allows the generation of datasets for different scenarios, including different types of triclustering patterns, pattern-specific datasets, datasets with overlapping triclusters, and datasets with noise and error values. These datasets enable to understand how the algorithms perform under different conditions and to identify issues and direction for improvement. G-Tric can also be used to generate synthetic datasets for real-world applications where the triclustering solutions are unknown, providing a way to evaluate the algorithm's results in a controlled setting. In this work, G-Tric is used to generate several datasets with hidden solutions to evaluate the selected triclustering algorithms, using the settings described in the following section, which align with the proposed in [35].

Synthetic data were generated with varying characteristics to simulate different scenarios in five evaluation experiments with the following settings:

1. The first experiment uses a real-valued baseline dataset with 500 rows, 10 columns, and 5 contexts, containing 40 triclusters evenly distributed among four types of triclustering patterns: order preserving on contexts, fully constant, fully additive, and fully multiplicative. The planted triclusters covered approximately 17% of the dataset.
2. The second experiment uses four datasets with the same features, each containing patterns of only one type. The patterns were applied to each combination of dimensions (X , Y , Z), specified in the first three columns of [Tables 3](#) and [4](#). The Order Preserving (OP) dataset had 30 triclusters (the maximum number of this type of triclusters the generator was able to plant), while the

Table 3

Results for the specific pattern on each dimension (Constant, Additive, and Multiplicative).

X	Y	Z	TriCluster	TCtricluster	δ -trimax	TriGen - LSL	TriGen - MSL	TriGen - MSR3D	
Constant	Constant	Constant	1.00 $\pm$ 0.000	0.49 $\pm$ 0.391	0.02 $\pm$ 0.070	0.74 $\pm$ 0.195	0.74 $\pm$ 0.195	0.29 $\pm$ 0.130	R
			0.985	0.982	0.967	0.999	0.999	0.997	CE
			0.087	0.081	0.301	0.180	0.183	0.160	RMS3
Constant	Constant	None	1.00 $\pm$ 0.000	0.57 $\pm$ 0.429	0.00 $\pm$ 0.000	0.68 $\pm$ 0.228	0.68 $\pm$ 0.228	0.34 $\pm$ 0.178	R
			0.987	0.982	1.000	0.999	0.999	0.997	CE
			0.085	0.083	0.000	0.169	0.169	0.155	RMS3
Constant	None	Constant	1.00 $\pm$ 0.000	0.65 $\pm$ 0.426	0.00 $\pm$ 0.000	0.87 $\pm$ 0.219	0.87 $\pm$ 0.219	0.40 $\pm$ 0.115	R
			0.988	0.982	1.000	0.999	0.999	0.997	CE
			0.035	0.036	0.000	0.182	0.182	0.163	RMS3
None	Constant	Constant	1.00 $\pm$ 0.000	0.77 $\pm$ 0.305	0.02 $\pm$ 0.047	0.77 $\pm$ 0.210	0.77 $\pm$ 0.210	0.31 $\pm$ 0.119	R
			0.984	0.972	0.980	0.999	0.999	0.997	CE
			0.093	0.092	0.263	0.186	0.182	0.159	RMS3
Constant	None	None	1.00 $\pm$ 0.000	0.75 $\pm$ 0.374	0.00 $\pm$ 0.000	0.87 $\pm$ 0.219	0.87 $\pm$ 0.219	0.36 $\pm$ 0.188	R
			0.986	0.976	1.000	0.999	0.999	0.997	CE
			0.034	0.033	0.000	0.189	0.187	0.169	RMS3
None	None	Constant	0.08 $\pm$ 0.033	0.06 $\pm$ 0.029	0.00 $\pm$ 0.000	0.82 $\pm$ 0.241	0.82 $\pm$ 0.241	0.38 $\pm$ 0.172	R
			0.999	0.998	1.000	0.999	0.999	0.997	CE
			0.031	0.031	0.000	0.188	0.188	0.172	RMS3
None	Constant	None	1.00 $\pm$ 0.000	0.51 $\pm$ 0.369	0.00 $\pm$ 0.000	0.85 $\pm$ 0.199	0.85 $\pm$ 0.199	0.42 $\pm$ 0.117	R
			0.986	0.981	1.000	0.999	0.999	0.996	CE
			0.092	0.100	0.000	0.197	0.196	0.168	RMS3
Additive	Additive	Additive	0.33 $\pm$ 0.080	0.19 $\pm$ 0.113	0.04 $\pm$ 0.089	0.76 $\pm$ 0.158	0.84 $\pm$ 0.174	0.26 $\pm$ 0.086	R
			0.994	0.992	0.956	0.999	0.999	0.996	CE
			0.057	0.059	0.160	0.193	0.188	0.173	RMS3
Additive	Additive	Constant	0.40 $\pm$ 0.052	0.29 $\pm$ 0.144	0.07 $\pm$ 0.212	0.77 $\pm$ 0.234	0.78 $\pm$ 0.236	0.24 $\pm$ 0.071	R
			0.994	0.990	0.925	0.999	0.999	0.997	CE
			0.062	0.064	0.125	0.189	0.190	0.164	RMS3
Additive	Constant	Additive	1.00 $\pm$ 0.000	0.54 $\pm$ 0.410	0.06 $\pm$ 0.190	0.68 $\pm$ 0.228	0.75 $\pm$ 0.225	0.26 $\pm$ 0.143	R
			0.982	0.977	0.937	0.999	0.999	0.996	CE
			0.093	0.089	0.120	0.189	0.191	0.162	RMS3
Constant	Additive	Additive	1.00 $\pm$ 0.000	0.45 $\pm$ 0.413	0.00 $\pm$ 0.000	0.71 $\pm$ 0.208	0.77 $\pm$ 0.251	0.27 $\pm$ 0.115	R
			0.986	0.984	1.000	0.999	0.999	0.997	CE
			0.041	0.043	0.000	0.180	0.183	0.154	RMS3
Additive	Constant	Constant	1.00 $\pm$ 0.000	0.53 $\pm$ 0.415	0.01 $\pm$ 0.025	0.72 $\pm$ 0.240	0.77 $\pm$ 0.251	0.23 $\pm$ 0.076	R
			0.984	0.980	0.991	0.999	0.999	0.997	CE
			0.079	0.075	0.062	0.205	0.204	0.155	RMS3
Constant	Constant	Additive	1.00 $\pm$ 0.000	0.48 $\pm$ 0.423	0.04 $\pm$ 0.139	0.77 $\pm$ 0.203	0.80 $\pm$ 0.219	0.24 $\pm$ 0.079	R
			0.985	0.982	0.952	0.999	0.999	0.997	CE
			0.073	0.073	0.109	0.188	0.190	0.156	RMS3
Constant	Additive	Constant	1.00 $\pm$ 0.000	0.71 $\pm$ 0.276	0.01 $\pm$ 0.022	0.75 $\pm$ 0.202	0.80 $\pm$ 0.219	0.26 $\pm$ 0.073	R
			0.983	0.971	0.992	0.999	0.999	0.996	CE
			0.036	0.040	0.060	0.194	0.195	0.163	RMS3
Multip.	Multip.	Multip.	1.00 $\pm$ 0.000	0.35 $\pm$ 0.360	0.00 $\pm$ 0.000	0.70 $\pm$ 0.219	0.70 $\pm$ 0.219	0.24 $\pm$ 0.083	R
			0.980	0.983	1.000	0.999	0.999	0.998	CE
			0.031	0.033	0.000	0.172	0.168	0.154	RMS3
Multip.	Multip.	Constant	1.00 $\pm$ 0.000	0.75 $\pm$ 0.380	0.00 $\pm$ 0.000	0.79 $\pm$ 0.192	0.79 $\pm$ 0.192	0.29 $\pm$ 0.084	R
			0.976	0.960	1.000	0.999	0.999	0.996	CE
			0.036	0.041	0.000	0.194	0.190	0.170	RMS3
Multip.	Constant	Multip.	1.00 $\pm$ 0.000	0.57 $\pm$ 0.437	0.00 $\pm$ 0.000	0.75 $\pm$ 0.225	0.75 $\pm$ 0.225	0.28 $\pm$ 0.095	R
			0.979	0.973	1.000	0.999	0.999	0.997	CE
			0.083	0.079	0.000	0.178	0.179	0.158	RMS3
Constant	Multip.	Multip.	1.00 $\pm$ 0.000	0.61 $\pm$ 0.444	0.00 $\pm$ 0.000	0.83 $\pm$ 0.222	0.83 $\pm$ 0.222	0.27 $\pm$ 0.087	R
			0.978	0.970	1.000	0.999	0.999	0.997	CE
			0.046	0.051	0.000	0.199	0.201	0.166	RMS3
Multip.	Constant	Constant	1.00 $\pm$ 0.000	0.61 $\pm$ 0.361	0.00 $\pm$ 0.000	0.85 $\pm$ 0.199	0.85 $\pm$ 0.199	0.33 $\pm$ 0.107	R
			0.978	0.970	1.000	0.999	0.999	0.996	CE
			0.100	0.101	0.000	0.190	0.188	0.170	RMS3
Constant	Constant	Multip.	1.00 $\pm$ 0.000	0.57 $\pm$ 0.432	0.00 $\pm$ 0.000	0.68 $\pm$ 0.216	0.68 $\pm$ 0.228	0.23 $\pm$ 0.106	R
			0.981	0.975	1.000	0.999	0.999	0.998	CE
			0.092	0.085	0.000	0.177	0.179	0.158	RMS3
Constant	Multip.	Constant	1.00 $\pm$ 0.000	0.29 $\pm$ 0.267	0.00 $\pm$ 0.000	0.72 $\pm$ 0.208	0.72 $\pm$ 0.208	0.28 $\pm$ 0.088	R
			0.979	0.987	1.000	0.999	0.999	0.997	CE
			0.037	0.043	0.000	0.180	0.180	0.158	RMS3

Table 4
Results for the specific pattern on each dimension (Order Preserving).

X	Y	Z	TriCluster	TCtricluster	δ -trimax	TriGen - LSL	TriGen - MSL	TriGen - MSR3D	
Order Pres.	None	None	0.05 ± 0.016	0.04 ± 0.019	0.00 ± 0.000	0.58 ± 0.180	0.72 ± 0.159	0.14 ± 0.066	R
			0.998	0.995	1.000	0.992	0.993	0.989	CE
			0.048	0.056	0.000	0.207	0.193	0.171	RMS3
None	Order Pres.	None	0.05 ± 0.016	0.03 ± 0.008	0.00 ± 0.000	0.58 ± 0.163	0.70 ± 0.146	0.13 ± 0.056	R
			0.997	0.997	1.000	0.992	0.993	0.989	CE
			0.048	0.046	0.000	0.201	0.190	0.162	RMS3
None	None	Order Pres.	0.06 ± 0.023	0.03 ± 0.012	0.00 ± 0.000	0.57 ± 0.116	0.70 ± 0.199	0.14 ± 0.056	R
			0.997	0.997	1.000	0.992	0.992	0.989	CE
			0.058	0.047	0.000	0.194	0.196	0.161	RMS3

Constant (C), Additive (A), and Multiplicative (M) datasets had 70 planted subspaces each. These datasets were used to study how the algorithms performed based on the dimension where coherence was applied. The datasets used in these first two experiments did not have overlapping features and had maximum quality, meaning there were no missing values, errors, or noise.

- The third experiment considers overlapping in the four pattern-specific datasets to evaluate how the performance deteriorates as triclusters blend together. The overlapping settings are described in Table 2.
- The fourth experiment considers noise and error values added to the original pattern-specific datasets to lower the coherence values of the planted triclusters. This tests how the algorithms' performance is affected by typical issues present in real-world data. The quality settings are summarized in Table 2. Originally we tried a setting with missing values, but we had to remove it since the assessed algorithms were unable to handle them.
- The last experiment considers contiguity in the four pattern-specific planted triclusters, specifically evaluating the ability of the algorithms to find triclusters with contiguity in the temporal dimension, which is a key topic in temporal triclustering. The properties defined for this dataset collection are described in Table 2.

5.3. Evaluation of results

The performance of the triclustering algorithms was evaluated using the three introduced metrics: Recoverability (R), Clustering Error (CE), and Revised Match Score 3D (RMS3).

Recoverability measures the proportion of planted triclusters that were correctly detected by the algorithm. A higher recoverability score indicates that the algorithm is better at detecting planted triclusters. CE measures the dissimilarity between the detected triclusters and the planted ones. A lower clustering error indicates that the detected triclusters are more similar to the planted ones. RMS3, on the other hand, evaluates the similarity in detail to all the triclustering dimensions and penalizes the non-matched elements. A poor similarity in one of the dimensions can negatively influence the metric value. The results for each algorithm were obtained by running the algorithms with the generated datasets. The results were then compared and analyzed to determine which algorithm performed best in terms of the three metrics. Specific assessments about the types of planted patterns were also performed to get more insights into the algorithm's behavior.

6. Results and discussion

6.1. Experiments with the first data collection (baseline)

The results of applying the selected triclustering algorithms to the first collection of datasets are presented in Fig. 3. TriGen (with MSL merit function) performed the best in terms of recovering the planted triclusters. The results of TriGen are in line with those obtained by the authors when comparing the three merit functions, with LSL performing better than the others [38]. However, TriCluster achieved the

second-best result in terms of recoverability, yielding a lower Clustering Error. This indicates that the method may be capable of finding a subset of the planted subspaces but not the entire set. This highlights the importance of considering the varying coherence types present in the dataset since each algorithm specializes in only some types. Therefore, it is not advisable to make premature conclusions about accuracy without further analysis.

To gain a deeper understanding of each algorithm's behavior, the use of ground truth data and extrinsic metrics enables an analysis of the impact of specific patterns on the performance of the algorithms. Fig. 4(A), (B), and (C) summarize the R, CE, and RMS3 scores of each algorithm based on subspaces with a single type of coherence. This allows for a more informed assessment of the success of each method in finding the target triclusters and differentiating between expected and unexpected results. As TriCluster is capable of detecting shifting and scaling patterns, it is expected to have good performance when retrieving Constant, Additive, and Multiplicative patterns. The method indeed achieved perfect scores on Constant and Multiplicative patterns, but failed to find the Additive ones. Further investigation may be necessary to understand why this is the case and whether the dimension in which coherence is applied yields significant impact on the results. Moreover, we could verify that TriGen was able to recover a higher number of the planted patterns, but, given the maximization of triclusters as one of the objectives, its performance can be degraded by a high clustering error.

6.2. Experiments with the second data collection

To further examine the performance of the algorithms, a second set of datasets were evaluated, with results summarized in Tables 3 and 4.

These results indicate generalized difficulties by δ -trimax and TriGen (MSR3D) in finding the planted triclusters with different types of coherence. TriCluster performs well in the majority of cases where the pattern is additive but struggles with fully additive or additive patterns in both the object and attribute dimensions. This could be due to the biclustering approach used in the method, which only detects shifting patterns in one dimension. Conversely, TriGen (using LSL and MSL merit functions) performs well in detecting constant patterns and has the best results with fully additive patterns. TriGen, given its multiobjective behavior, arguably appears to be the most versatile algorithm able to recover patterns from all the planted types. However, its higher clustering errors reveal its difficulty in controlling precision (extraction of non-planted data elements).

Analyzing the matching quality per dimension provides insights into how each algorithm deals with different patterns and what dimensions they search for coherence. This type of analysis also pinpoints any limitations in the methods, which may not be immediately evident without using synthetic data with ground truth.

In summary, the results show that TriCluster and TriGen (MSL and MSR3D) perform well in terms of recoverability, while TriGen (MSL and MSR3D) have low clustering error and RMS3 scores. TCtricluster and δ -Trimax show lower performance across all three metrics. In particular, for the Order Preserving pattern type, TriGen (MSL, MSR3D, and LSL)

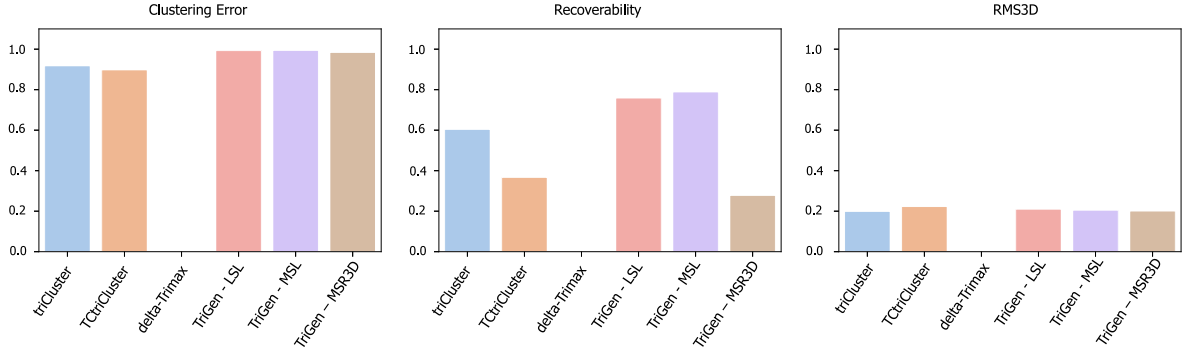


Fig. 3. Recoverability, Clustering Error, and RMS3 scores for each algorithm - first collection. δ -Trimax did not find any tricluster.

performs better in terms of recoverability and RMS3, while TriCluster and TCriCluster show moderate to low performance.

Overall, TriCluster achieved high recoverability for most of the specific patterns, with the understandable exception of Order Preserving patterns, which is (by design) unprepared to detect. TriGen (with MSL function) was found to be the most versatile algorithm, able to recover triclusters across the explored range of patterns.

6.3. Experiments with the third data collection

The third experiment in this comparison examines the effect of overlapping on the performance of each algorithm. The datasets used in this experiment were derived from the baseline dataset and modified to have the overlapping properties outlined in Table 2. The key differences from the baseline data include: 40% of the planted triclusters overlap with another subspace in a pairwise manner, meaning there are no sets of three or more triclusters overlapping. Each pair of overlapping triclusters shares a maximum of 50% of their elements relative to the smallest tricluster, and there is no limit on the amount of overlap permitted along each dimension. Fig. 4(D), (E), and (F) resume the performance results of the triclustering algorithms on each of the four datasets, one per pattern type, with overlapping. The results remained similar regarding all the assessed metrics, with Order Preserving settings continuing with poor performance. This is somewhat expected since none of the evaluated algorithms was designed to identify this type of pattern. However, TriGen (MSL) achieved high Recoverability with a small decrease compared with the baseline (0.75 vs. 0.82).

Nonetheless, the results are promising when investigating algorithmic behaviors with other patterns. On the Multiplicative dataset, the CE score was not affected in the majority of the algorithms, having decreased only for TCriCluster (0.82 vs. 0.79). Regarding the other patterns, this score was not affected.

Recoverability, in general, was not also affected. TriGen (LSL) increased the R score (0.73 vs. 0.74), looking for Additive patterns. This might be a consequence of the plaid coherency assumed (Additive), which, as opposed to what happened with the other patterns, does not make the overlapped region follow the same coherency type of the original triclusters.

In short, TriCluster, in fact, exhibits difficulties handling overlapping regions, as pinpointed by the authors [3]. Something highlighted as difficult that would be difficult to evaluate in the absence of data's ground truth.

6.4. Experiments with the fourth data collection

A crucial aspect impacting an algorithm's performance is the quality of the data it is tested on. Poor data quality makes it harder to identify coherent subspaces that can form triclusters. By using synthetic data with ground truth for analysis, we can determine which algorithms are better suited to deal with varying forms and degrees of noise, or at least

can minimize its effect. The final group of datasets was based on the baseline dataset but with a degree of added noise and gross errors both in the background and individual triclusters.

The results are depicted in Fig. 5(G), (H), and (I) show that the ongoing underperformance of δ -trimax in recovering triclusters in the Additive and Multiplicative datasets can be attributed to the search for patterns with an MSR3D below a specific delta. This approach may lead to a high number of false positives due to the lack of statistical significance guarantees. Additionally, some false negatives may occur because of how the MSR combines contributions from different dimensions.

All algorithms, except TriGen, saw a decline in performance due to the added noise. TriGen remained relatively stable in terms of recoverability but also showed high clustering error, meaning that the triclusters found were larger than expected, as indicated by lower RMS3 values. Results on the algorithms' recoverability measure also showed poor performance, with a few small improvements in some metrics driven by the recovery of larger triclusters or solutions rather than the algorithms' ability to retrieve the subspaces accurately. None of the algorithms was able to effectively deal with varying forms and degrees of noise, or minimize its effect.

These observations are not surprising, as none of the evaluated algorithms were explicitly designed to tolerate varying degrees of noise in spite of their ability to accommodate it. However, TriGen was found to be the only algorithm that remained unaffected due to its multi-objective nature with fitness functions that evaluate the interactions between objects. The presence of noisy elements within the data poses a significant challenge to the discovery of relevant triclusters as real-world patterns are generally affected by noise. Therefore, it is crucial to tackle this limitation in future research to enable the effective discovery of such triclusters.

6.5. Experiments with the fifth data collection

The temporal aspect requires the application of specific constraints to the subspaces being investigated. In particular, the contiguity of tricluster's time points when focusing on subsets of objects, features, and times to ensure the stability of coherence along time. The latest data collection followed the same properties as the previous one, but this time, we only focused on triclusters with contiguity in the temporal dimension.

In this latest experiment, TriGen performed well in terms of recoverability, as shown in Fig. 5(J), (K), and (L). In particular, TriCluster and TCriCluster, which showed similar recoverability (with TCriCluster having a slight improvement), are competitive with TriGen's performance in the Multiplicative dataset. TCriCluster was specifically designed to find temporally contiguous triclusters [2], and, as the results show, it remains the best option for achieving this goal by obtaining the lowest clustering error and the highest RMS3.

The ability to find contiguity in temporal patterns while preserving specific coherence, although particularly addressed by the TCriCluster

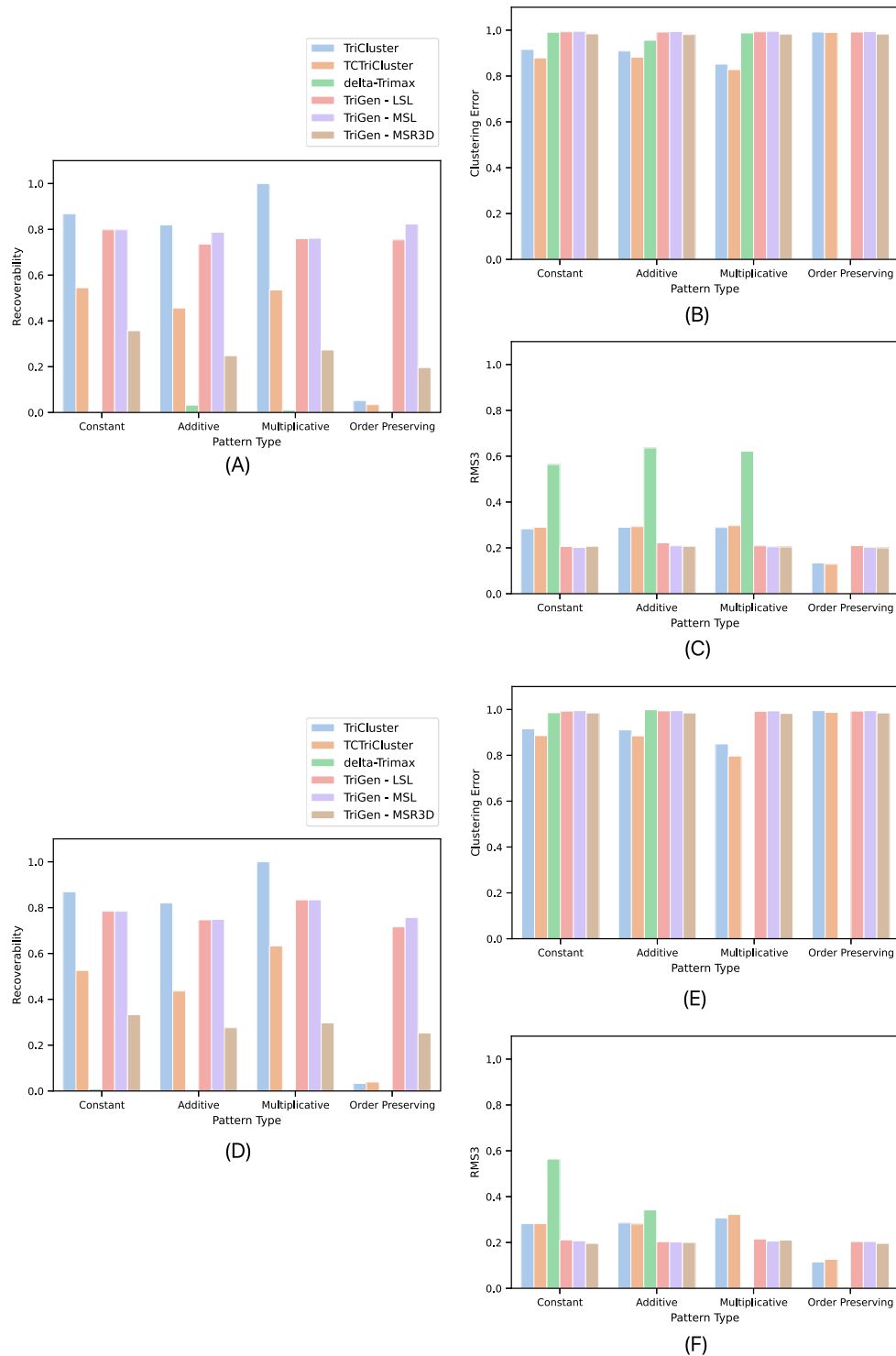


Fig. 4. Recoverability, Clustering Error, and RMS3 scores for each pattern type by algorithm. Figures A–C report to the first/second collections, and Figures D–F report to the third collection. δ -Trimax did not recover any tricluster in A–B (Order Preserving), D–F (Order Preserving).

algorithm, remains an open-door challenge given the pivotal role of temporal progression patterns in longitudinal studies across several data and application domains [2].

7. Conclusion and future research directions

The emergence of triclustering algorithms and their notable applications has triggered the need for their solid evaluation and comparison as a way of guiding the selection of the most effective approaches for

a given data context and specific application. This study provides a significant contribution by establishing a benchmark for assessing temporal triclustering algorithms, allowing for fairer comparisons in the future. We performed a comprehensive assessment of four algorithms using five synthetic data collections with planted triclusters (ground truth), evaluating the capabilities of the algorithms to find triclusters with different types of pattern coherence in 3W datasets with varying characteristics. The results pinpointed the strengths and limitations of the assessed algorithms in three-way temporal data analysis.

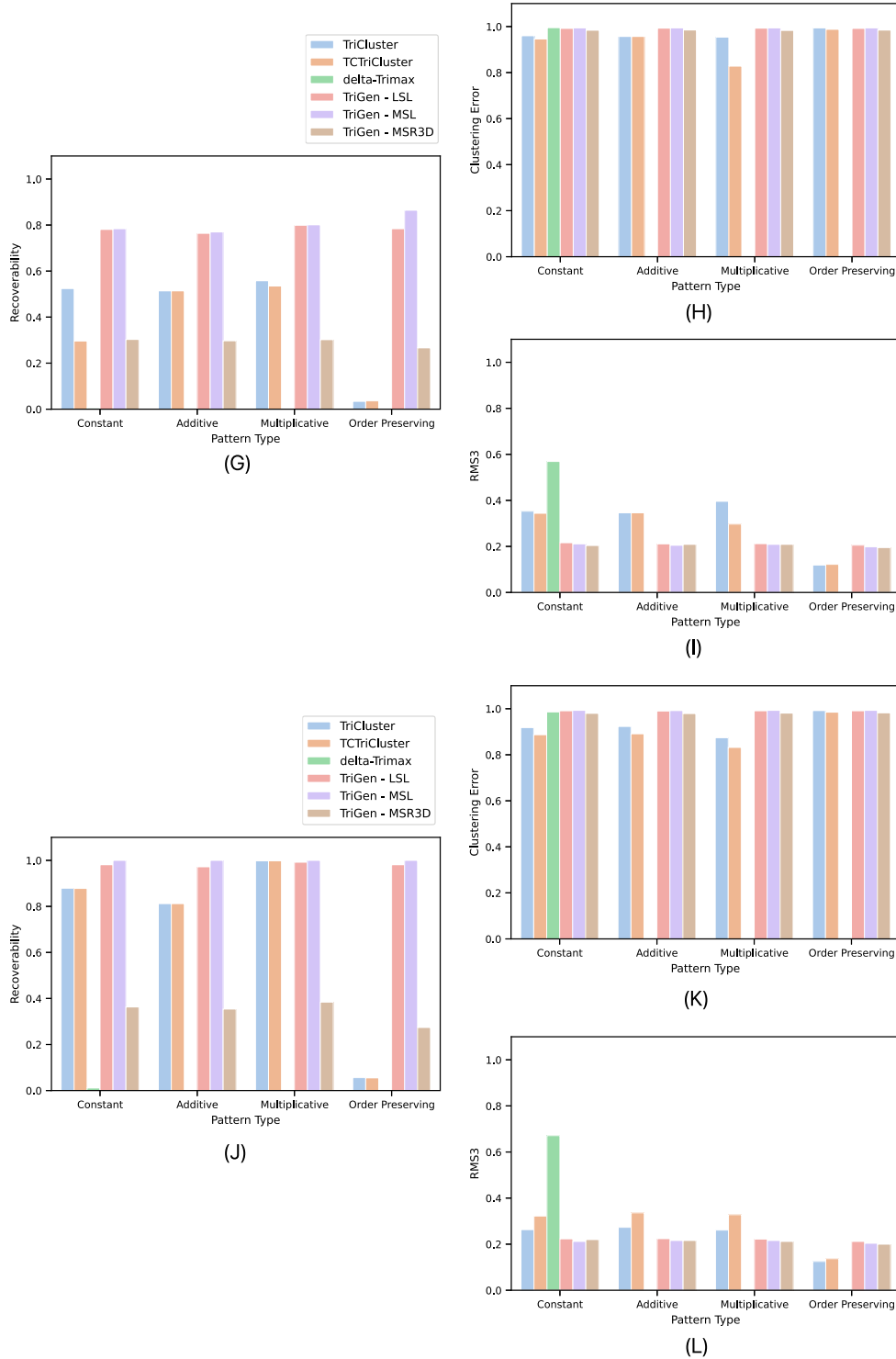


Fig. 5. Recoverability, Clustering Error, and RMS3 scores for each pattern type by algorithm. Figures G–I report to the fourth collection and Figures J–L report to the fifth collection. δ -Trimax did not recover any tricluster in G–I (Additive, Multiplicative) and J–L (Constant, Multiplicative, Order Preserving).

Despite the limited number of algorithms evaluated, this study is the first comprehensive assessment of state-of-the-art triclustering algorithms for pattern discovery in three-way temporal data. By following the proposed evaluation methodology for normative assessments, future researchers can use this study as a reference point to compare and improve their algorithms. We further strongly recommend prospective authors to ensure that source code is made readily available to facilitate continuous improvements and proper comparisons.

This study represents a noteworthy advance in the triclustering research field as it provides a clear baseline for future developments to build upon. Future research directions based on the limitations of the assessed algorithms include: (1) addressing temporal misalignments through enhanced methods; (2) investigating triclustering techniques designed explicitly for sparse 3D data; (3) preparing triclustering methods to unveil triclustering solutions that account for varying homogeneity criteria within triclusters; (4) improving methods to handle different types and degrees of noise in temporal data; (5) revisiting the design

of triclustering algorithms in light of the highlighted difficulties in handling the recoverability-precision trade-off, including revisiting objectives in optimization-driven approaches and hyperparameterizations in heuristic approaches.

CRedit authorship contribution statement

Diogo F. Soares: Conceptualization, Investigation, Methodology, Software, Writing – original draft, Writing – review & editing. **Rui Henriques:** Conceptualization, Methodology, Supervision, Validation, Writing – review & editing. **Sara C. Madeira:** Funding acquisition, Methodology, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and materials availability

The generated datasets and the source code run for the experiments were made available at: <https://github.com/dfmsoares/triclustering-algorithms-assessment>.

Acknowledgments

This work was supported by Fundação para a Ciência e a Tecnologia (FCT) through project AlPALS ref. PTDC/CCI-CIF/4613/2020 (<https://doi.org/10.54499/PTDC/CCI-CIF/4613/2020>), LASIGE Research Unit, ref. UIDB/00408/2020 (<https://doi.org/10.54499/UIDB/00408/2020>) and ref. UIDP/00408/2020 (<https://doi.org/10.54499/UIDP/00408/2020>), INESC-ID Research Unit, ref. UIDB/50021/2020 (<https://doi.org/10.54499/UIDB/50021/2020>), and a PhD research scholarship (ref. 2020.05100.BD) to DFS.

References

- [1] R. Henriques, S.C. Madeira, Triclustering algorithms for three-dimensional data analysis: A comprehensive survey, *ACM Comput. Surv.* 51 (5) (2018) 95, <http://dx.doi.org/10.1145/3195833>.
- [2] D.F. Soares, R. Henriques, M. Gromicho, M. de Carvalho, S.C. Madeira, Triclustering-based classification of longitudinal data for prognostic prediction: targeting relevant clinical endpoints in amyotrophic lateral sclerosis, *Sci. Rep.* 13 (1) (2023) 6182.
- [3] L. Zhao, M.J. Zaki, TRICLUSTER: An effective algorithm for mining coherent clusters in 3D microarray data, in: *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*, SIGMOD '05, ACM, New York, NY, USA, 2005, pp. 694–705.
- [4] H. Jiang, S. Zhou, J. Guan, Y. Zheng, gTRICLUSTER: a more general and effective 3D clustering algorithm for gene-sample-time microarray data, in: *International Workshop on Data Mining for Biomedical Applications*, Springer, 2006, pp. 48–59.
- [5] G. Wang, L. Yin, Y. Zhao, K. Mao, Efficiently mining time-delayed gene expression patterns, *IEEE Trans. Syst. Man Cybern. B* 40 (2) (2009) 400–411.
- [6] D.F. Soares, R. Henriques, M. Gromicho, M. de Carvalho, S.C. Madeira, Learning prognostic models using a mixture of biclustering and triclustering: predicting the need for non-invasive ventilation in Amyotrophic lateral sclerosis, *Journal of Biomedical Informatics* 134 (2022) 104172.
- [7] H.A. Ahmed, P. Mahanta, D.K. Bhattacharyya, J.K. Kalita, A. Ghosh, Intersected coexpressed subcube miner: An effective triclustering algorithm, in: *2011 World Congress on Information and Communication Technologies*, 2011, pp. 846–851, <http://dx.doi.org/10.1109/WICT.2011.6141358>.
- [8] A.B. Tchagang, S. Phan, F. Famili, H. Shearer, P. Fobert, Y. Huang, J. Zou, D. Huang, A. Cutler, Z. Liu, et al., Mining biological information from 3D short time-series gene expression data: the OPTricluster algorithm, *BMC Bioinformatics* 13 (1) (2012) 54.
- [9] T. Kakati, H.A. Ahmed, D.K. Bhattacharyya, J.K. Kalita, THD-Tricluster: A robust triclustering technique and its application in condition specific change analysis in HIV-1 progression data, *Comput. Biol. Chem.* 75 (2018) 154–167.
- [10] K. Mandal, R. Sarmah, D.K. Bhattacharyya, POPTric: Pathway-based Order Preserving Triclustering for gene sample time data analysis, *Expert Syst. Appl.* 192 (2022) 116336.
- [11] A. Li, D. Tuck, An effective tri-clustering algorithm combining expression data with gene regulation information, *Gene Regul. Syst. Biol.* 3 (2009) GRSB—S1150.
- [12] A. Bhar, M. Haubrock, A. Mukhopadhyay, U. Maulik, S. Bandyopadhyay, E. Wingender, δ -TRIMAX: Extracting triclusters and analysing coregulation in time series gene expression data, 2012, pp. 165–177, http://dx.doi.org/10.1007/978-3-642-33122-0_13.
- [13] T. Siswantining, N. Saputra, D. Sarwinda, H.S. Al-Ash, Triclustering discovery using the δ -trimax method on microarray gene expression data, *Symmetry* 13 (3) (2021) 437.
- [14] W.A. Daeng, T. Siswantining, A. Bustamam, P. Anki, δ -TRIMAX method with silhouette coefficient on microarray gene expression data for early detection of heart failure, in: *2022 5th International Conference on Information and Communications Technology, ICOIACT, IEEE*, 2022, pp. 412–416.
- [15] K. Sim, Z. Aug, V. Gopalkrishnan, Discovering correlated subspace clusters in 3D continuous-valued data, in: *2010 IEEE International Conference on Data Mining, IEEE*, 2010, pp. 471–480, <http://dx.doi.org/10.1109/ICDM.2010.19>.
- [16] X. Xu, Y. Lu, K.-L. Tan, A.K.H. Tung, Finding time-lagged 3D clusters, in: *2009 IEEE 25th International Conference on Data Engineering, IEEE*, 2009, pp. 445–456, <http://dx.doi.org/10.1109/ICDE.2009.80>.
- [17] L. Ji, K.-L. Tan, A.K.H. Tung, Mining frequent closed cubes in 3D datasets, in: *Proceedings of the 32nd International Conference on Very Large Data Bases, VLDB Endowment*, 2006, pp. 811–822.
- [18] Y.-C. Liu, C.-H. Lee, W.-C. Chen, J.W. Shin, H.-H. Hsu, V.S. Tseng, A novel method for mining temporally dependent association rules in three-dimensional microarray datasets, in: *2010 International Computer Symposium, ICS2010, IEEE*, 2010, pp. 759–764, <http://dx.doi.org/10.1109/COMPSYM.2010.5685410>.
- [19] Z. Hu, R. Bhatnagar, Discovery of versatile temporal subspace patterns in 3-D datasets, in: *2011 IEEE 11th International Conference on Data Mining, IEEE*, 2011, pp. 1092–1097, <http://dx.doi.org/10.1109/ICDM.2011.56>.
- [20] K.C. Mondal, M. Ghosh, R. Fajriyah, A. Roy, Introducing suffix forest for mining tri-clusters from time-series data, *Innov. Syst. Softw. Eng.* (2022) 1–23.
- [21] D.I. Ignatov, S.O. Kuznetsov, Frequent itemset mining for clustering near duplicate web documents, in: *International Conference on Conceptual Structures, Springer*, 2009, pp. 185–200.
- [22] B.G. Mirkin, A.V. Kramarenko, Approximate bicluster and tricluster boxes in the analysis of binary data, in: *International Workshop on Rough Sets, Fuzzy Sets, Data Mining, and Granular-Soft Computing*, Springer, 2011, pp. 248–256.
- [23] M. Yurov, D.I. Ignatov, Turning krimp into a triclustering technique on sets of attribute-condition pairs that compress, in: *International Joint Conference on Rough Sets*, Springer, 2017, pp. 558–569.
- [24] D.I. Ignatov, D.V. Gnatyshak, S.O. Kuznetsov, B.G. Mirkin, Triadic formal concept analysis and triclustering: searching for optimal patterns, *Mach. Learn.* 101 (1–3) (2015) 271–302, <http://dx.doi.org/10.1007/s10994-015-5487-y>.
- [25] A. Houari, W. Ayadi, S. Ben Yahia, A new FCA-based method for identifying biclusters in gene expression data, *Int. J. Mach. Learn. Cybern.* 9 (11) (2018) 1879–1893, <http://dx.doi.org/10.1007/s13042-018-0794-9>.
- [26] G.K. Gerber, R.D. Dowell, T.S. Jaakkola, D.K. Gifford, Automated discovery of functional generality of human gene expression programs, *PLoS Comput. Biol.* 3 (8) (2007) e148, <http://dx.doi.org/10.1371/journal.pcbi.0030148>.
- [27] X. Li, Y. Ye, M. Ng, Q. Wu, MultiFacTV: module detection from higher-order time series biological data, *BMC Genomics* 14 (4) (2013) S2.
- [28] S. Mankad, G. Michailidis, Biclustering three-dimensional data arrays with plaid models, *J. Comput. Graph. Statist.* 23 (4) (2014) 943–965, <http://dx.doi.org/10.1080/10618600.2013.851608>.
- [29] D. Amar, D. Yekutieli, A. Maron-Katz, T. Hendler, R. Shamir, A hierarchical Bayesian model for flexible module discovery in three-way time-series data, *Bioinformatics* 31 (12) (2015) i17–i26.
- [30] J. Liu, Z. Li, X. Hu, Y. Chen, Multi-objective evolutionary algorithm for mining 3D clusters in gene-sample-time microarray data, in: *2008 IEEE International Conference on Granular Computing, IEEE*, 2008, pp. 442–447.
- [31] D. Gutiérrez-Avilés, C. Rubio-Escudero, F. Martínez-Álvarez, J. Riquelme, TriGen: A genetic algorithm to mine triclusters in temporal gene expression data, *Neurocomputing* 132 (2014) 42–53, <http://dx.doi.org/10.1016/j.neucom.2013.03.061>.
- [32] A. Bhar, M. Haubrock, A. Mukhopadhyay, E. Wingender, Multiobjective triclustering of time-series transcriptome data reveals key genes of biological processes, *BMC Bioinformatics* 16 (1) (2015) 200, <http://dx.doi.org/10.1186/s12859-015-0635-8>.
- [33] D. Gutiérrez-Avilés, C. Rubio-Escudero, MSL: a measure to evaluate three-dimensional patterns in gene expression data, *Evol. Bioinform.* 11 (2015) EBO—S25822.
- [34] D. Gutiérrez-Avilés, C. Rubio-Escudero, LSL: A new measure to evaluate triclusters, in: *2014 IEEE International Conference on Bioinformatics and Biomedicine, BIBM, IEEE*, 2014, pp. 30–37.
- [35] J. Lobo, R. Henriques, S.C. Madeira, G-Tric: generating three-way synthetic datasets with triclustering solutions, *BMC Bioinformatics* 22 (1) (2021) 16, <http://dx.doi.org/10.1186/s12859-020-03925-4>.
- [36] D. Horta, R.J. Campello, Similarity measures for comparing biclusterings, *IEEE/ACM Trans. Comput. Biol. Bioinform.* 11 (5) (2014) 942–954, <http://dx.doi.org/10.1109/TCBB.2014.2325016>.

- [37] A. Patrikainen, M. Meila, Comparing subspace clusterings, *IEEE Trans. Knowl. Data Eng.* 18 (7) (2006) 902–916.
- [38] D. Gutiérrez-Avilés, R. Giráldez, F.J. Gil-Cumbreras, C. Rubio-Escudero, TRIQ: a new method to evaluate triclusters, *BioData Min.* 11 (2018) 1–29.

Diogo F. Soares has an M.Sc. in Data Science from Ciências ULisboa and is pursuing a Ph.D. in Computer Science at the same institution. He is currently an invited Teaching Assistant at Ciências ULisboa and a Student Researcher at LASIGE. His current main research interests are in Data Mining and Unsupervised Learning Algorithms, focusing concretely on the problem of finding coherent temporal patterns in temporal three-way data with triclustering in health and biomedical problems.

Rui Henriques is a professor at Instituto Superior Técnico, University of Lisbon, where he lectures courses in the domains of Machine Learning, Artificial Intelligence, and Health Systems. Doctorate (2016) by Instituto Superior Técnico, Rui had wide-exposure to real projects in different sectors as a business analyst at McKinsey and the head of the AI area at Tekever. Rui is also an Associate Researcher at INESC-ID, where he coordinates multidisciplinary R&D projects. His research contributions

cross advances in machine learning with real-world applications in biomedical and urban domains. Rui has authored reference papers in pattern discovery, biclustering and triclustering, including Triclustering Algorithms for Three-Dimensional Data Analysis: A Comprehensive Survey. Rui has been ascribed two scientific career awards in 2020 and 2022.

Sara C. Madeira has a Ph.D. in Computer Science and Engineering from Técnico - ULisboa and is currently an Associate Professor at Ciências - ULisboa and a Senior Researcher at LASIGE, where she coordinates the Data and Systems Intelligence Research Line of Excellence (RLE) and is also a member of the Health and Biomedical Informatics RLE. Her research interests fall in the broad area of Data Science, focusing on machine learning, in particular biclustering and triclustering algorithms, and their applications to complex data analysis in health and biomedical informatics. In this context, she coordinated/coordinated national and international projects targeting applications in neurodegenerative diseases and published several highly cited papers, being the first author of Biclustering Algorithms for Biological Data Analysis: A survey, published in 2004, considered an ESI Hot Paper in Computer Science in November 2006, and now with more than 2500 citations, and the last author of Triclustering Algorithms for Three-Dimensional Data Analysis: A Comprehensive survey, published in the end of 2018. She received the best researcher award in Computer Science and Engineering from ULisboa in 2021.